

Practical No 05

Name : Pranjal somatkar roll no. : B 48

AIM: Data Visualization with Matplotlib: Data visualization on a dataset using matplotlib

Theory

Data Visualization with Matplotlib & Seaborn

1. Importance of Data Visualization in Data Analysis & Role in EDA

Introduction

Data visualization is the graphical representation of data using charts, graphs, and plots. It transforms raw data into a visual format, making it easier to understand patterns, trends, and relationships.

Importance of Data Visualization

1. Simplifies Complex Data

Large datasets can be difficult to interpret in raw form. Visualization converts them into easy-to-understand visuals.

Example:

Raw Data	Visualization
Numbers in rows	Graph showing trends

2. Helps in Better Decision Making

Visual insights help businesses and researchers make quick and accurate decisions.

3. Identifies Patterns & Trends

Graphs reveal:

- Increasing/decreasing trends
- Seasonal patterns
- Correlations

4. Detects Outliers

Unusual values are easily spotted using plots like box plots.

5. Improves Communication

Visuals make it easier to present findings to non-technical audiences.

Role in Exploratory Data Analysis (EDA)

EDA is the first step in analyzing data to understand its structure.

How Visualization Helps in EDA

Task in EDA	Visualization Used	Purpose
Distribution analysis	Histogram	Shows frequency
Relationship analysis	Scatter plot	Shows correlation
Trend analysis	Line plot	Shows change over time
Outlier detection	Box plot	Detects extreme values
Multi-variable analysis	Pair plot	Shows relationships between multiple variables

Example

```
import matplotlib.pyplot as plt
import seaborn as sns

data = [10, 20, 30, 40, 50]

plt.plot(data)
plt.title("Simple Line Plot")
plt.show()
```

2. Difference Between Matplotlib and Other Visualization Libraries

Introduction

Matplotlib is a foundational plotting library in Python, while other libraries like Seaborn, Plotly, and Pandas visualization provide additional features.

Comparison Table

Feature	Matplotlib	Seaborn	Plotly
Level	Low-level	High-level	High-level
Ease of Use	Moderate	Easy	Easy
Customization	Very high	Moderate	High
Interactivity	No	No	Yes
Built-in Themes	Limited	Advanced	Advanced
Use Case	Basic & advanced plots	Statistical visualization	Interactive dashboards

Explanation

Matplotlib

- Base library for plotting
- Highly customizable
- Requires more code

Seaborn

- Built on Matplotlib
- Better for statistical plots
- Attractive default styles

Plotly

- Interactive graphs
- Used in web applications

3. Purpose of pyplot Module in Matplotlib

Introduction

`pyplot` is a module in Matplotlib that provides functions for creating plots easily.

Main Functions of pyplot

Function	Purpose
<code>plt.plot()</code>	Line plot
<code>plt.scatter()</code>	Scatter plot
<code>plt.bar()</code>	Bar chart
<code>plt.hist()</code>	Histogram
<code>plt.boxplot()</code>	Box plot
<code>plt.title()</code>	Add title
<code>plt.xlabel()</code>	X-axis label
<code>plt.ylabel()</code>	Y-axis label
<code>plt.show()</code>	Display plot

Why pyplot is Important

- Simplifies plotting
- Provides MATLAB-like interface
- Easy for beginners

Example

```
import matplotlib.pyplot as plt

x = [1, 2, 3]
y = [10, 20, 30]

plt.plot(x, y)
plt.title("Line Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

4. Process of Creating a Basic Line Plot

Step-by-Step Process

Step 1: Import Library

```
import matplotlib.pyplot as plt
```

Step 2: Prepare Data

```
x = [1, 2, 3, 4]
y = [10, 20, 25, 30]
```

Step 3: Create Plot

```
plt.plot(x, y)
```

Step 4: Add Labels & Title

```
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Basic Line Plot")
```

Step 5: Display Plot

```
plt.show()
```

Full Example

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 20, 25, 30]

plt.plot(x, y)
plt.xlabel("Time")
plt.ylabel("Values")
plt.title("Line Plot Example")
plt.show()
```

Output

A graph showing a line connecting the points (1,10), (2,20), (3,25), (4,30).

5. Importance of Customization in Data Visualization

Introduction

Customization makes graphs more meaningful, readable, and visually appealing.

Why Customization is Important

1. Improves Readability

Clear labels and titles help understand the graph easily.

2. Enhances Presentation

Useful for reports and presentations.

3. Highlights Important Data

Colors and styles emphasize key points.

4. Makes Graph Attractive

Better visuals improve user engagement.

Customization Options in Matplotlib

1. Titles and Labels

```
plt.title("Graph Title")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
```

2. Colors and Styles

```
plt.plot(x, y, color='red', linestyle='--')
```

3. Markers

```
plt.plot(x, y, marker='o')
```

4. Grid

```
plt.grid(True)
```

5. Legends

```
plt.legend(["Data Line"])
```

6. Figure Size

```
plt.figure(figsize=(8,5))
```

7. Axis Limits

```
plt.xlim(0, 10)
plt.ylim(0, 50)
```

Example with Customization

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
```

```

y = [10, 20, 25, 30]

plt.figure(figsize=(8,5))
plt.plot(x, y, color='blue', linestyle='--', marker='o')
plt.title("Customized Line Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.show()

```

▼ Data Visualization on Tips Dataset

1. Import Libraries & Load Dataset

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
df = pd.read_csv('/content/tip.csv')

# Check data
print(df.head())

```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

2. Scatter Plot

```

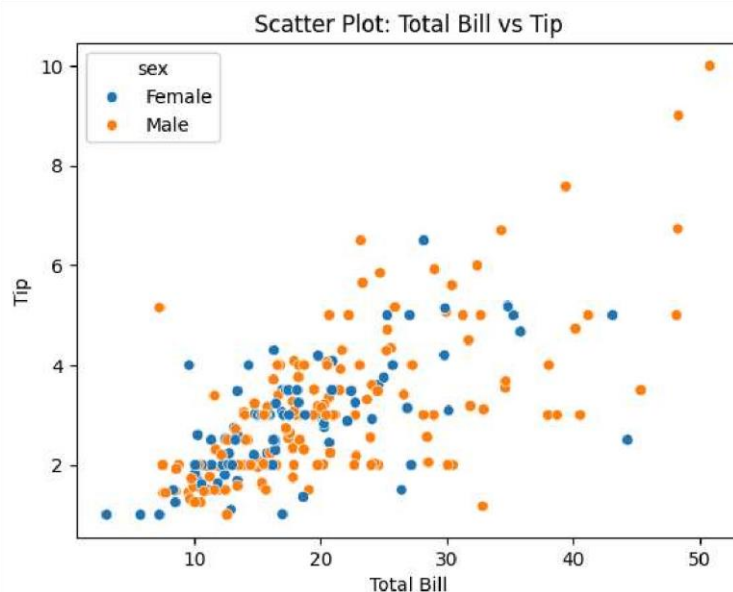
plt.figure()

sns.scatterplot(x='total_bill', y='tip', hue='sex', data=df)

plt.title("Scatter Plot: Total Bill vs Tip")
plt.xlabel("Total Bill")
plt.ylabel("Tip")

plt.show()

```



3. Line Plot

```

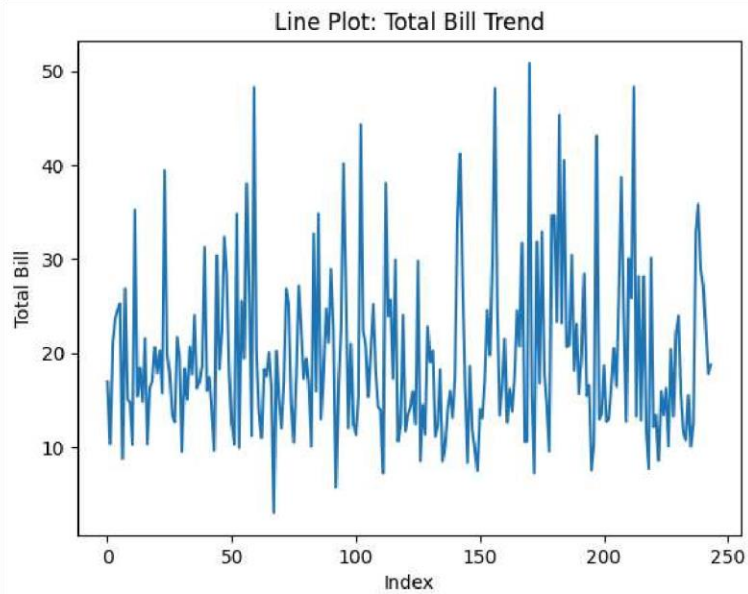
plt.figure()

sns.lineplot(x=df.index, y='total_bill', data=df)

```

```
plt.title("Line Plot: Total Bill Trend")
plt.xlabel("Index")
plt.ylabel("Total Bill")

plt.show()
```



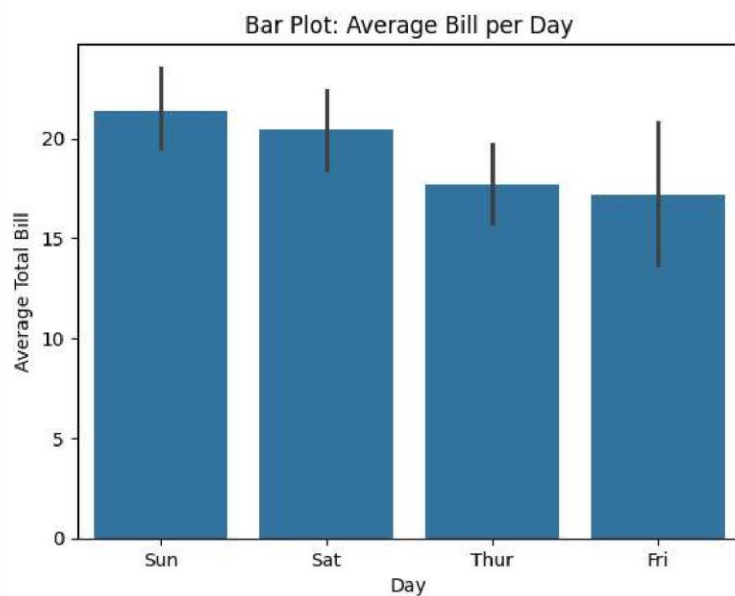
4. Bar Plot

```
plt.figure()

sns.barplot(x='day', y='total_bill', data=df)

plt.title("Bar Plot: Average Bill per Day")
plt.xlabel("Day")
plt.ylabel("Average Total Bill")

plt.show()
```



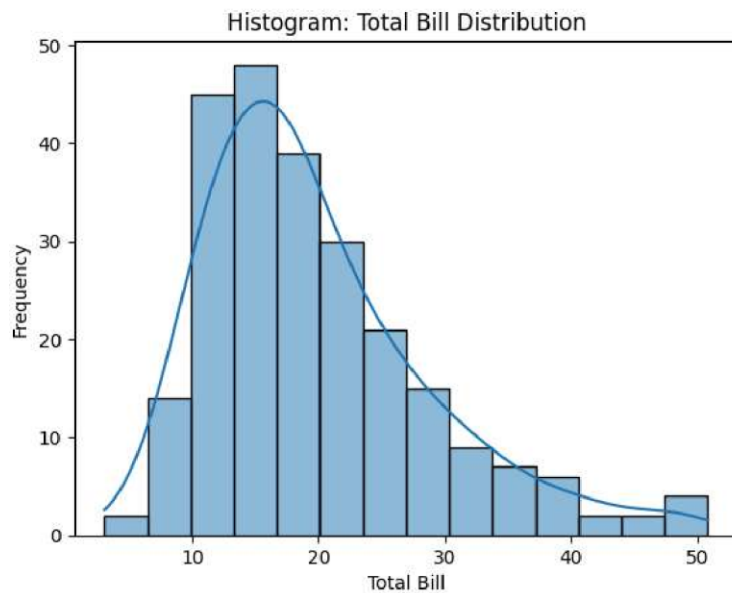
5. Histogram

```
plt.figure()

sns.histplot(df['total_bill'], kde=True)

plt.title("Histogram: Total Bill Distribution")
plt.xlabel("Total Bill")
plt.ylabel("Frequency")

plt.show()
```



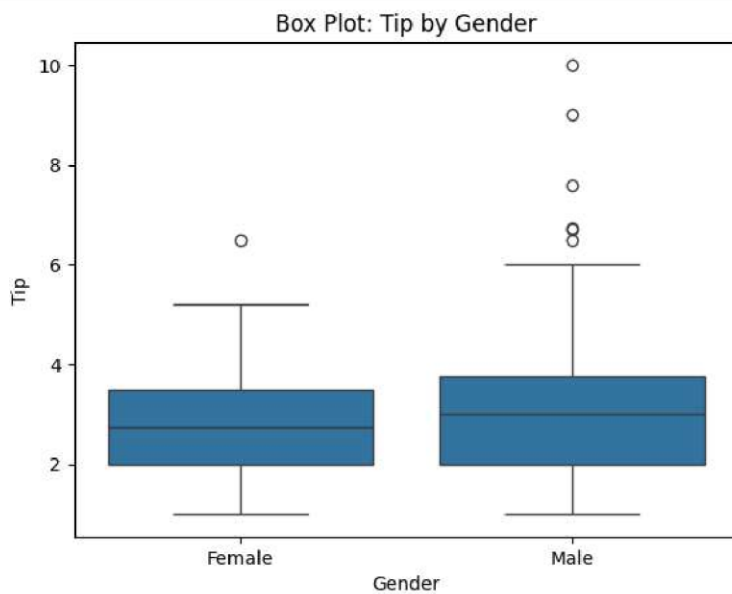
6. Box Plot

```
plt.figure()

sns.boxplot(x='sex', y='tip', data=df)

plt.title("Box Plot: Tip by Gender")
plt.xlabel("Gender")
plt.ylabel("Tip")

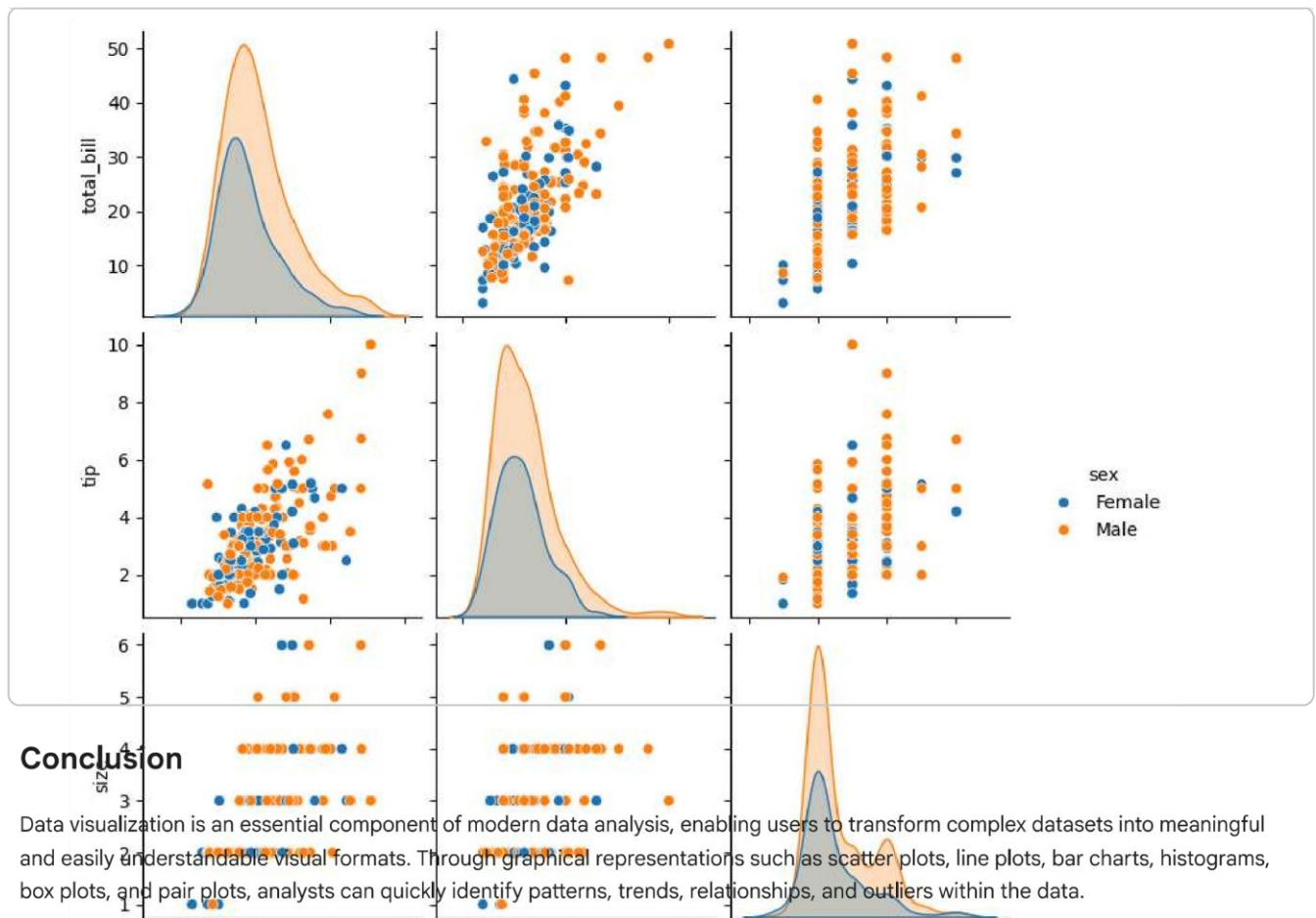
plt.show()
```



7. Pair Plot

```
sns.pairplot(df, hue='sex')

plt.show()
```



Conclusion

Data visualization is an essential component of modern data analysis, enabling users to transform complex datasets into meaningful and easily understandable visual formats. Through graphical representations such as scatter plots, line plots, bar charts, histograms, box plots, and pair plots, analysts can quickly identify patterns, trends, relationships, and outliers within the data.

Libraries like **Matplotlib** and **Seaborn** play a crucial role in this process. Matplotlib provides a strong foundation with extensive customization options, while Seaborn enhances visualization with more advanced statistical features and aesthetically pleasing designs. The `pyp1ot` module further simplifies the process by offering an easy-to-use interface for creating a wide variety of plots.

Data visualization is especially important in **Exploratory Data Analysis (EDA)**, where it helps in understanding the structure and characteristics of the dataset before applying any advanced analytical or machine learning techniques. It allows analysts to make informed decisions, detect errors, and gain valuable insights efficiently.

Moreover, customization options such as colors, labels, markers, grids, and styles improve the clarity and effectiveness of visualizations, making them more impactful for presentations and reports.

In conclusion, mastering data visualization tools like Matplotlib and Seaborn is essential for anyone working with data, as it not only enhances analytical capabilities but also improves the communication of insights in a clear, concise, and visually appealing manner.